Step 3 — Pause, Resume, and the CLI — Solution

> Back to the step · Hint 1 · Hint 2

Story: this compiles the graph with a real checkpointer, runs it, and handles the pause properly — on escalation the run genuinely stops, a human replies, and the graph resumes from exactly where it paused. **If not:** `interrupt()` would have no checkpointer to pause against, and "escalate to a human" would never actually wait for one to respond.

Sample help docs (`data/help_docs/`)

```
# data/help_docs/refunds.md
Refunds are issued within 14 days of returning the item. Keep your receipt
and contact support with your order number to start a refund.
```

```
# data/help_docs/password.md
To reset your password, click "Forgot password" on the login page and follow
the emailed link. The link expires in one hour.
```

```
# data/help_docs/shipping.md
Orders ship in 2 business days. Track your shipment from the Orders page using
the tracking number in your confirmation email.
```

(Leave account-deletion, legal, and billing-dispute questions *uncovered*, so they escalate.)

main.py — answer, or pause for a human

```
# main.py
import sys

from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command

from src.config import load_config
from src.graph import build_graph
from src.models.llm import get_model
```

```
from src.rag.help_store import build_help_store
from src.state import start_state


def main():
    if len(sys.argv) < 2:
        print('usage: python main.py "your question"')
        return
    question = sys.argv[1]

    config = load_config()
    model = get_model(config)
    collection = build_help_store()
    # MemorySaver for a demo; SqliteSaver(...) to survive a restart
    app = build_graph(model, collection, config.confidence_cutoff).compile(
        checkpointer=MemorySaver())

    thread = {"configurable": {"thread_id": "cli-session"}}
    result = app.invoke(start_state(question), thread)

    paused = app.get_state(thread).next
    if paused:                                      # it escalated and paused
        print("DeflectDesk: I'm not confident I can answer that from our "
              "help docs.\n  -> Escalating to a human agent...")
        human = input("  [human agent, type a reply]: ")
        result = app.invoke(Command(resume=human), thread)
        print(f"DeflectDesk (via human): {result['answer']}   [escalated]")
    else:
        print(f"DeflectDesk: {result['answer']}")


if __name__ == "__main__":
    main()
```

How the pause works

- Compiling with a **checkpointer** is what lets `interrupt()` pause: the state is saved after every node, so a stopped run can be resumed later with the same `thread_id`.
- After `invoke`, **`app.get_state(thread).next` is non-empty** when the graph paused at the escalate node (it's `("escalate",)`). That's how you detect an escalation.
- **`Command(resume=human)`** feeds the human's reply back into the paused `interrupt()`, and the graph runs to the end with `answer = human`, `escalated = True`.
- Swap `MemorySaver` for `SqliteSaver` and the pause survives a process restart — a human can answer hours later.

The integration test (real model + RAG + graph)

```
# tests/integration/test_live_support.py
# Real model + RAG + graph. Run from the repo root:
#   python -m tests.integration.test_live_support
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command

from src.config import load_config
from src.graph import build_graph
from src.models.llm import get_model
from src.rag.help_store import build_help_store
from src.state import start_state


def build_app():
    config = load_config()
    model = get_model(config)
    collection = build_help_store()
    graph = build_graph(model, collection, config.confidence_cutoff)
    return graph.compile(checkpointer=MemorySaver())


def test_confident_question_is_answered():
    app = build_app()
    thread = {"configurable": {"thread_id": "t-answer"}}
    result = app.invoke(start_state("How do refunds work?"), thread)
    ok = not app.get_state(thread).next and "refund" in result["answer"].lower()
    result1 = "WRONG"
    if ok:
        result1 = "OK"
    print(f"case 1 (covered question answered) -> {result1}")


def test_offtopic_escalates_and_resumes():
    app = build_app()
    thread = {"configurable": {"thread_id": "t-escalate"}}
    app.invoke(start_state("Delete my account permanently"), thread)
    paused = bool(app.get_state(thread).next)
    result = app.invoke(Command(resume="A human will handle this."), thread)
    ok = paused and result["escalated"] is True
    status = "WRONG"
    if ok:
        status = "OK"
    print(f"case 2 (off-topic escalates + resumes) -> {status}")
```

```
if __name__ == "__main__":
    test_confident_question_is_answered()
    test_offtopic_escalates_and_resumes()
```

Expected run:

```
$ python main.py "How do refunds work?"
DeflectDesk: Refunds are issued within 14 days of returning the item...

$ python main.py "Delete my account permanently"
DeflectDesk: I'm not confident I can answer that from our help docs.
  -> Escalating to a human agent...
  [human agent, type a reply]: A support agent will verify your identity.
DeflectDesk (via human): A support agent will verify your identity.
[escalated]
```

That's the whole project: a bot that answers from your docs when the match is confident, and genuinely hands off to a human — pausing and resuming — when it isn't. The honest escalation is the feature, and `interrupt()` + a checkpointer is what makes it real.